King Fahd University of Petroleum and Minerals
College of Computing and Mathematics

Information & Computer Science Department



ICS 202
Data Structures and Algorithms

Unit 03
Linked Lists

Term 261

Reading Assignment

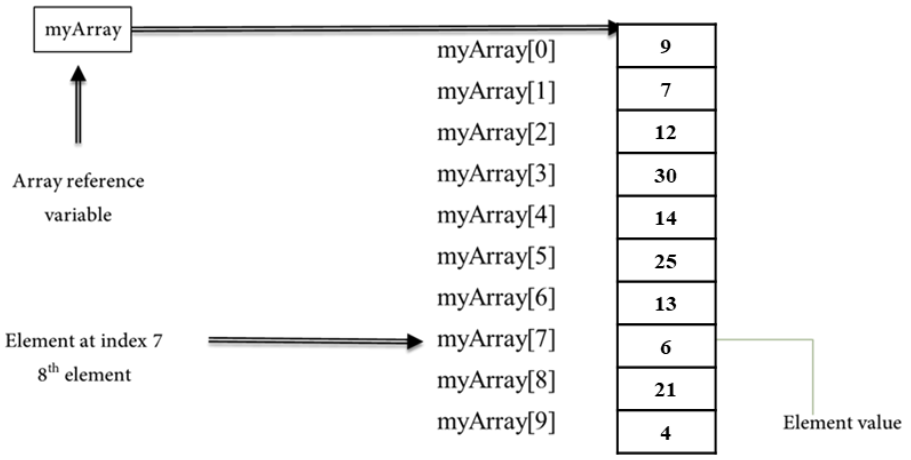
- "Data Structures and Algorithms in Java", 4th Edition, Adam Drozdek, Cengage Learning, ISBN 978-9814239233
  - Chapter 3 (Sections 1 – 3)
  - Sections 3.4-3.9 are not included.

Objectives

- Discuss the following topics:
  - Singly Linked Lists
  - Doubly Linked Lists
  - Circular Singly-linked Lists
  - Circular Doubly-linked Lists

Introduction

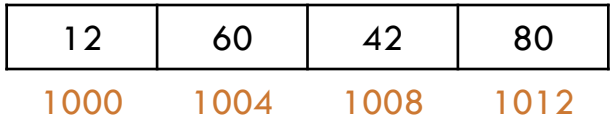
- An array is a sequence of values of the same type stored in consecutive memory locations.. `int[] myArray = new int[10];`
- It has at least two limitations:
  - Fixed size: to insert new elements, create a new array and then copy.
  - Inserting/deleting an item:
    - Shift some other items in the array.
  - Insert/Delete at the end
  - Insert/Delete at the beginning
  - Insert/Delete at position i



0	1	2	3	4	5	6	7	8	9
A	H	C	Z	X	A	V	N		

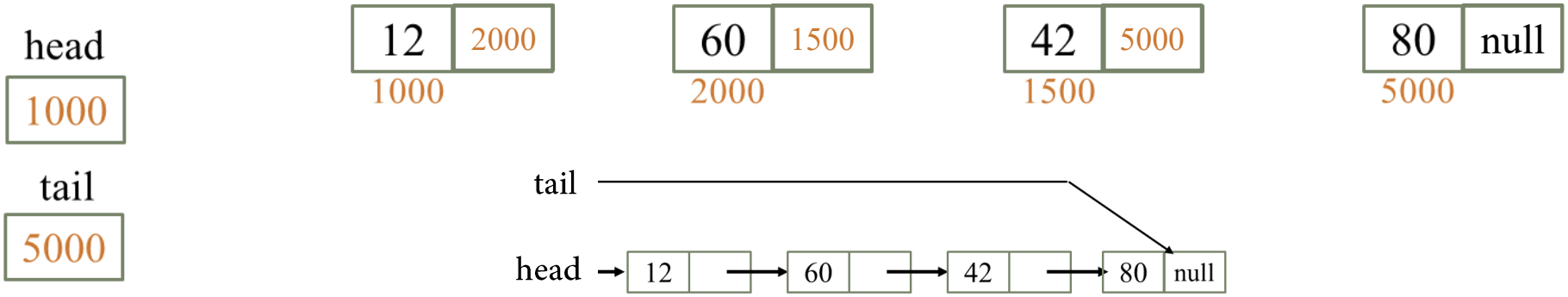
Example

- We want to store 4 numbers: 12, 60, 42, 80



  - Array: elements are stored in consecutive memory locations.
    - FAST random access: Indexing requires a single arithmetic calculation, independent of array length. Address ($A[i]$)= Base Address + (i × Element Size)
    - Memory-efficient

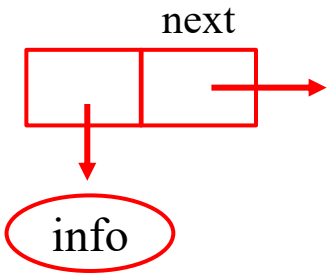
  - Linked list: elements are scattered in the memory and are connected.
    - Assume the elements are stored in the following memory locations 1000, 2000, 1500 and 5000 respectively.

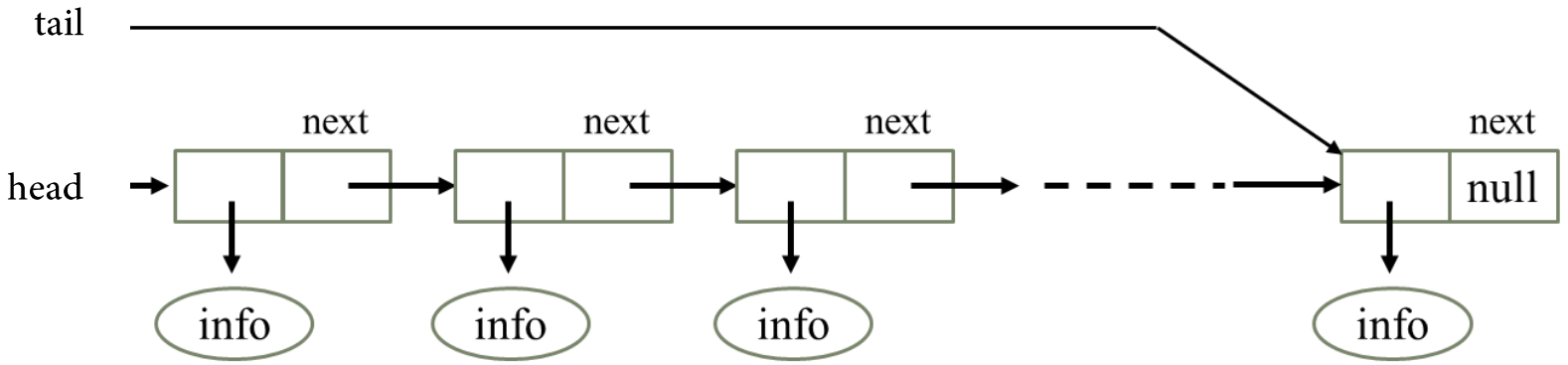


What is a Singly-linked list?

- A singly linked list (SLL) is a dynamic data structure consisting of a sequence of nodes(elements), forming a linear ordering.
- Each node stores:
  - info: (data object)
  - Next: Reference (i.e., address) to the next node

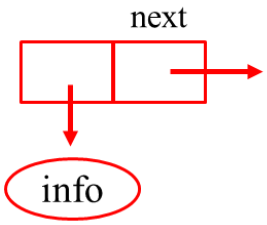
Node:





Node Representation

- In our representation, a node has:
  - info: (data object) and next: address to the next node




```
private class SLLNode<T> {
    private T info;
    private SLLNode<T> next;

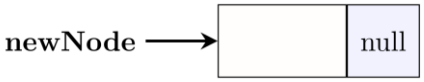
    public SLLNode() {
        this(null,null);
    }

    public SLLNode(T e) {
        this(e,null);
    }

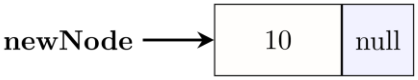
    public SLLNode(T e, SLLNode<T> ptr) {
        info = e; next = ptr;
    }

}
```

SLLNode<T> newNode = new SLLNode<T>();

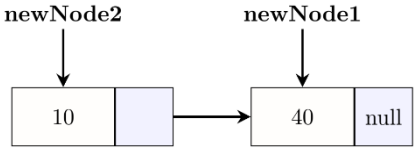


SLLNode<T> newNode = new SLLNode<T>(10);



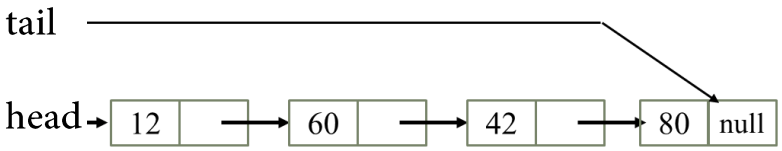
SLLNode<T> newNode1 = new SLLNode<T>(40);

SLLNode<T> newNode2 = new SLLNode<T>(10, newNode1);



Example 1

- Create the following list by creating SLLNode objects?




```
private class SLLNode<T> {
    private T info;
    private SLLNode<T> next;

    public SLLNode() {
        this(null,null);
    }

    public SLLNode(T e) {
        this(e,null);
    }

    public SLLNode(T e, SLLNode<T> ptr) {
        info = e; next = ptr;
    }

}
```

```
SLLNode <Integer> node1 = new SLLNode <>(12);
SLLNode <Integer> node2 = new SLLNode <>(60);
node1.next = node2;


SLLNode <Integer> node4 = new SLLNode <>(80);
SLLNode <Integer> node3 = new SLLNode <>(42, node4);
node2.next = node3;


SLLNode <Integer> head = node1;
SLLNode <Integer> tail = node4;
```

Linked List Representation

```
public class SLL<T> {

    // Internal Node class

    private class SLLNode<T> {
        private T info;
        private SLLNode<T> next;

        public SLLNode() {
            this(null,null);
        }

        public SLLNode(T e) {
            this(e,null);
        }

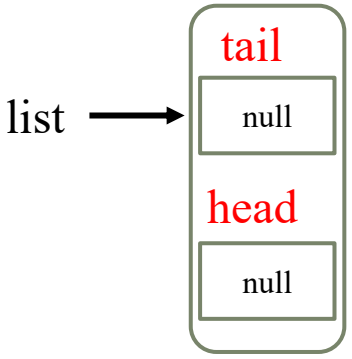
        public SLLNode(T e, SLLNode<T> ptr) {
            info = e; next = ptr;
        }

    }

    private SLLNode<T> head, tail;

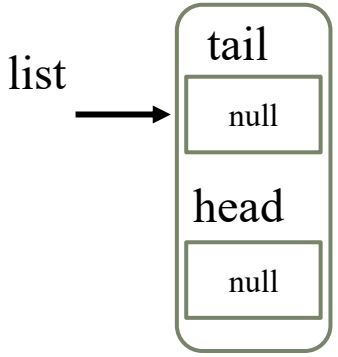
    public SLL() {
        head = tail = null;
    }
    public boolean isEmpty() {
        return head == null;
    }
```

- In our representation, a linked list has both head and tail references.



List Creation

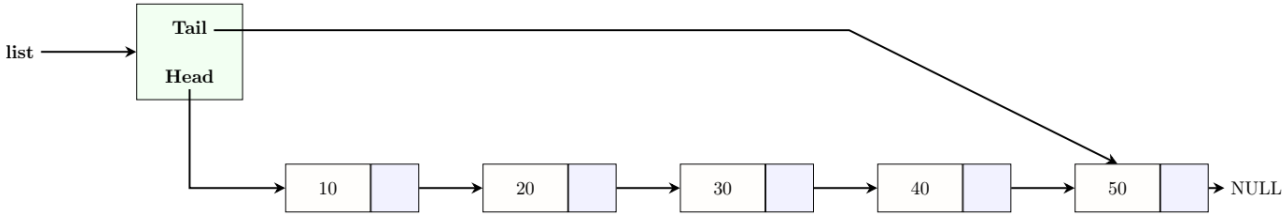
- An example of creating a linked list:




```
SLL<Integer> list = new SLL<>();
```

- Once created, elements can be inserted into the list using either the addToHead or addToTail methods:

```
for(int i = 10; i <= 50; i = i + 10)
    list.addToTail(i);
```



Insertion at the beginning (Prepend)

```
public void addToHead(T e) {

1       SLLNode<T> newNode = new SLLNode<T>(e);

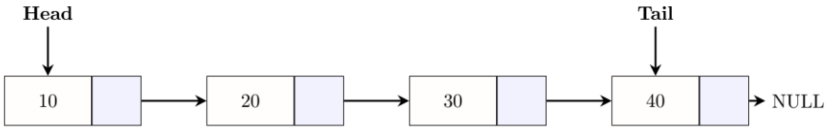
        if (isEmpty() ){
4           tail = newNode;
            head = newNode;
        }
        else {
2           newNode.next = head;
3           head = newNode;
        }

}
```

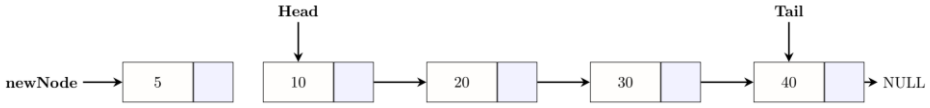
- When inserting a new node, we have two cases:
  - the list is not empty
  - the list is empty

Steps to Insert at Head (with empty list check):

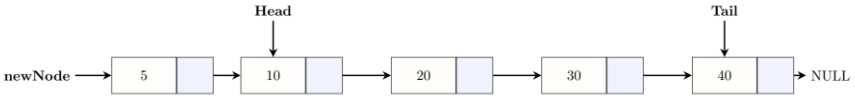
1. **Create a new node** with the desired value.
2. **If the list is empty (head is `null`):**
   - Set both `head` and `tail` pointers to the new node.
3. **Otherwise (list is not empty):**
   - Set the `next` of the new node to point to the current head.
   - Update the head pointer to point to the new node.



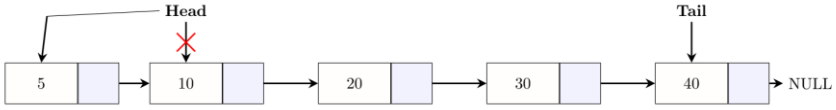
1



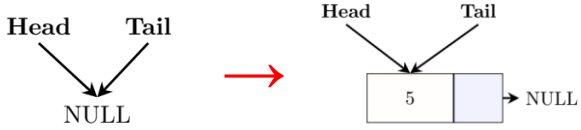
2



3



4



Complexity is

Insertion at the end (Append)

```
public void addToTail(T e) {

    SLLNode<T> newNode = new SLLNode<>(e);

    if (isEmpty()) {
        head = newNode;
        tail = newNode;
    }
    else {
        tail.next = newNode;
        tail =  newNode;
    }

}
```

1

4

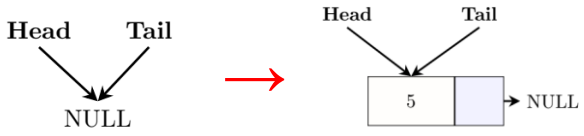
2

3

Steps to Insert at Tail (with empty list check):

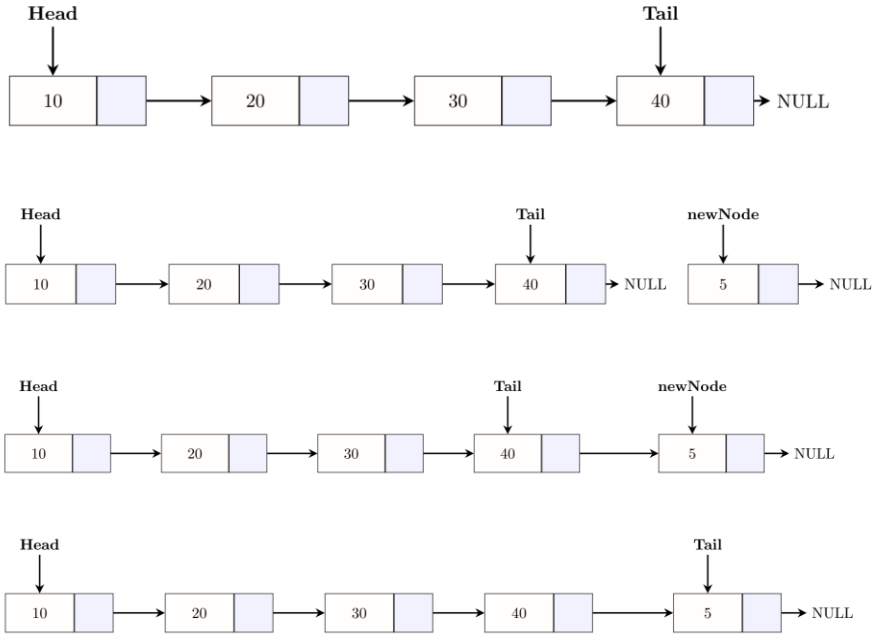
1. **Create a new node** with the desired value.
2. **Set the** `next` **of the new node to** `null` (since it will be the last node).
3. **If the list is empty (head is** `null` **):**
   - Set both `head` and `tail` pointers to the new node.
4. **Otherwise (list is not empty):**
   - Set the current tail's `next` pointer to the new node.
   - Update the tail pointer to point to the new node.

4



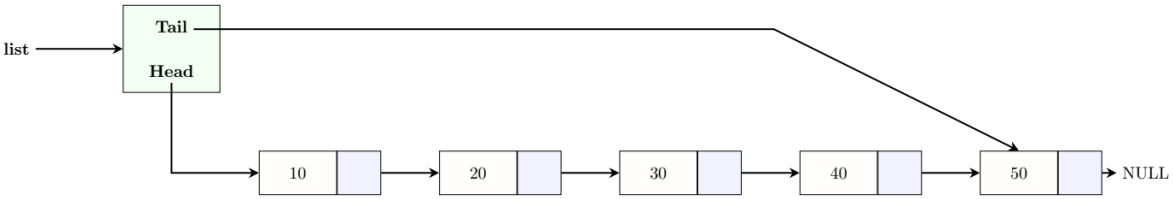
- When inserting a new node, we have two cases:
  - the list is not empty
  - the list is empty or

Complexity is



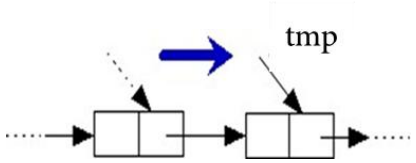
SLL Traversal

- Traversal means visiting each node in the list, one by one, usually to read or process the data stored in each node.



- Examples:
  - Printing a linked list
  - Searching a linked list
  - Performing operations like sum, max, min, or modify data

- Key Points
  - Always start from **head**
  - Move to **next** node in each step
  - Stop when you reach **the end or until the traversal condition is satisfied**



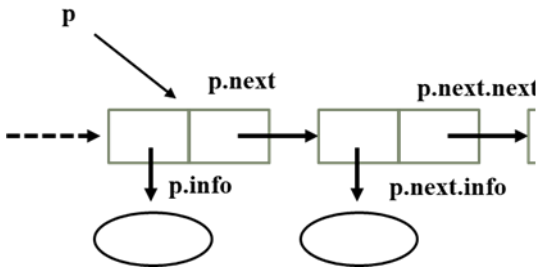

```
SLLNode<T> tmp = head;

while(tmp != null){

    // process tmp.info

    tmp = tmp.next;
}
```

- The following reference relationships are useful in searching



Searching & Printing

- Example:
  - Searching a linked list
  - Printing a linked list

```
public String toString() {
    if(isEmpty())
        return "[ ]";
    String str = "[ ";
    SLLNode<T> tmp = head;
    while(tmp != null){
        str += tmp.info + "   "
        tmp = tmp.next;
    }
    return str+"]";
}
```

O(n)

```
public boolean find(T e) {
    if(isEmpty())
        return false;
    SLLNode<T> tmp = head;
    while(tmp != null){
        if(tmp.info.equals(e))
            return true;
        tmp = tmp.next;
    }
    return false;
}
```

O(n)

$\Theta(n)$ in the worst case

$\Theta(1)$ in the best case

Insertion after a node with data e which is not the last node

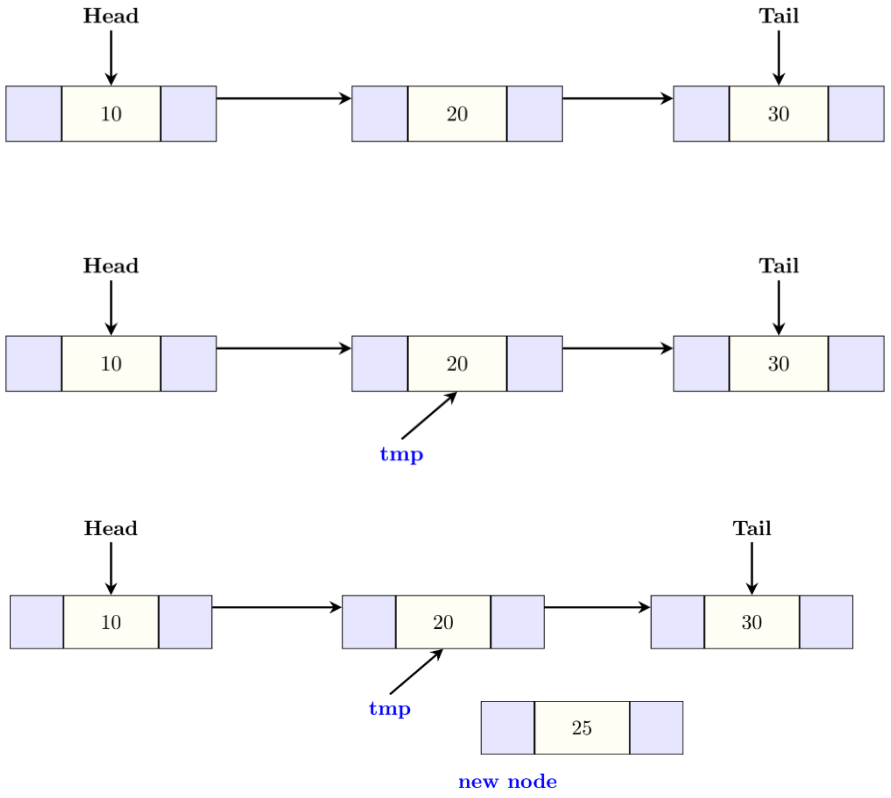
Assume node with data e1 exists and is not the last node.

```
public void insertAfter(T e, T e1){
    // Assume node with data e1 exists
    // and is not the last node
    SLLNode<T> newNode = new SLLNode<T>(e);
    SLLNode<T> tmp = head;

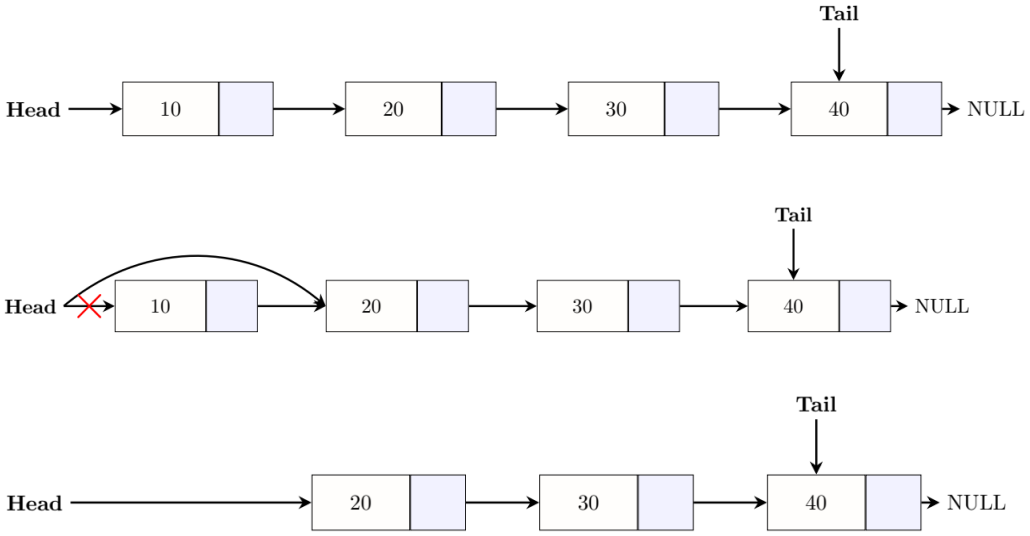
    while(!e1.equals(tmp.info))
        tmp = tmp.next;

    newNode.next = tmp.next;
    tmp.next = newNode;
}
```

Complexity is



Deletion – Deleting First Element



Steps to Delete from Head

1. **Check if the list is empty**
   - If `head == null`, there's nothing to delete. Stop.
2. **Check if the list has only one node**
   - If `head.next == null`, set `head = null` and `tail = null`.
3. **Otherwise (more than one node)**
   - Move the head pointer to the next node: `head = head.next`

```
public T deleteFromHead() {

    if (isEmpty())
        return null;

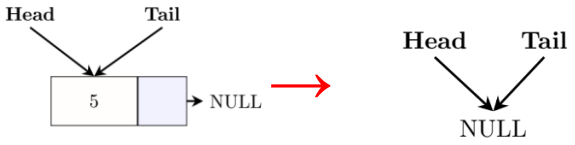
    T e = head.info;

2   if (head == tail)
        head = tail = null;
    else
1       head = head.next;

    return e;
}
```

Complexity is

One node 2



- When deleting a node, we have three cases:
  - the list is empty
  - the list has more than one node
  - the list has only one node

Deletion – Deleting Last node

```
public T deleteFromTail() {

    if (isEmpty())
        return null;

    T el = tail.info;

    if (head == tail)
        head = tail = null;

    else {

        SLLNode<T> tmp = head;

        while (tmp.next != tail)
            tmp = tmp.next;

        tail = tmp;
        tail.next = null;
    }

    return el;
}
```

1

2

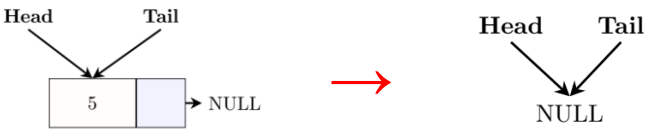
3

4

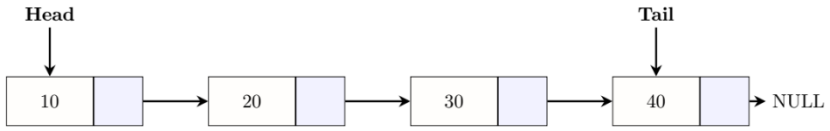
Complexity is

Steps to Delete from Tail (Singly Linked List):

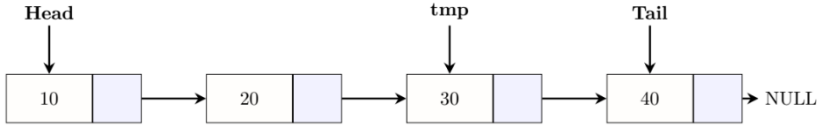
1. **Check if the list is empty (head is null):**
   - If yes, there's nothing to delete.
2. **If the list has only one node (head == tail):**
   - Set `head = null` and `tail = null` (list becomes empty).
3. **Otherwise, traverse the list to find the second-to-last node:**
   - Start from `head` and move forward until you reach the node whose `next` is the tail.
4. **Set the `next` pointer of the second-to-last node to null.**
5. **Update the tail pointer to point to the second-to-last node.**



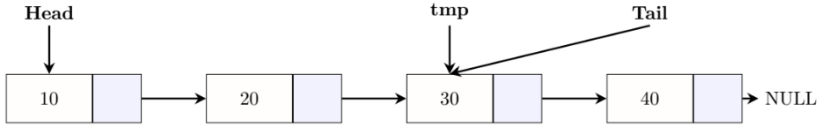
1



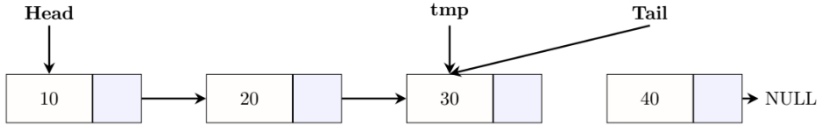
2

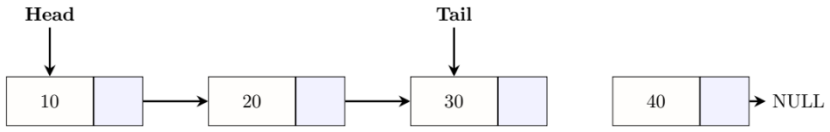


3



4





Deletion – Deleting first occurrence of an element

```java
public boolean delete(T e) {  // delete the node with element e
    if (isEmpty()) return false;

    // Case 1: only one node in the list
    if (head == tail && e.equals(head.info)) {
        head = tail = null;
        return true;
    }

    // Case 2: e is in the head node (more than one node)
    if (e.equals(head.info)) {
        head = head.next;
        return true;
    }

    // Case 3: e is in a non-head node
    SLLNode<T> prev = head, tmp = head.next;
    while (tmp != null && !tmp.info.equals(e)) {
        prev = tmp;
        tmp = tmp.next;
    }

    if (tmp == null) return false; // e not found

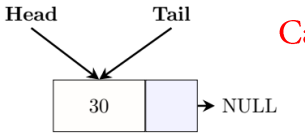
    // delete tmp
    prev.next = tmp.next;
    if (tmp == tail)
        tail = prev;

    return true;
}
```

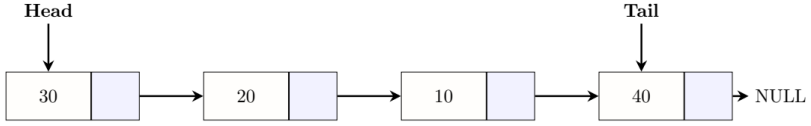
Complexity is

// if empty
//if only one node in the list;
//if more than one node on the list and el is in the head node;
// if more than one node in the list and el is in a non-head node
// if el is in the last node

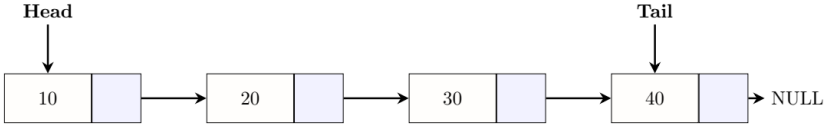
Case 1: delete from head (One node)



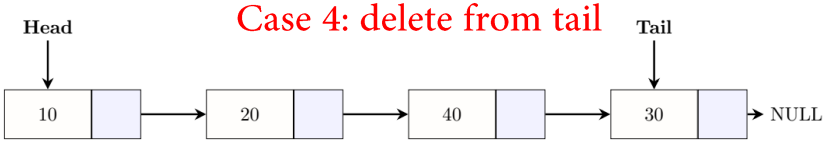
Case 2: delete from the head (many nodes)



Case 3: delete from middle

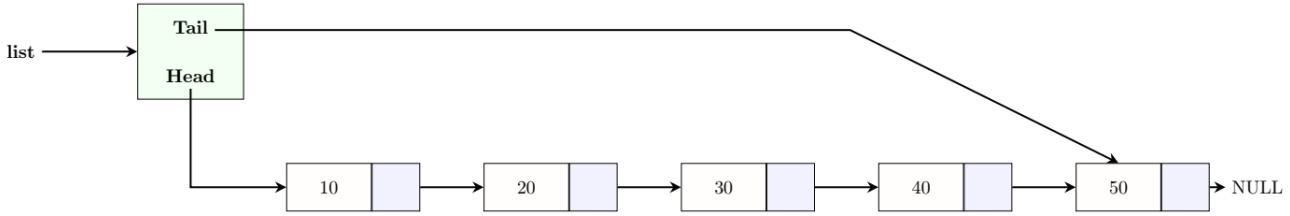


Case 4: delete from tail



Representation: Space Requirements

- A singly linked list stores:
  - **Data** for each node
  - A **pointer/reference** to the next node



- Number of references in the list and space required

Reference	Total
head	1
tail	1
next	n
info	n

So, the memory required grows **linearly** with the number of nodes.

$$\text{Total } Space = n \times \left(info\ size + reference\ size\right) \Rightarrow O(n)$$

The space complexity of a singly linked list with is O(n)

Singly-linked lists vs. 1D-arrays

ID-array	Singly-linked list
Fixed size: Resizing is expensive	Dynamic size
Insertions and Deletions are inefficient: Elements are usually shifted	Insertions and Deletions are efficient: No shifting
Random access i.e., efficient indexing	No random access → Not suitable for operations requiring accessing elements by index such as sorting
No memory waste if the array is full or almost full; otherwise, may result in much memory waste.	Extra storage needed for references; however, uses exactly as much memory as it needs
Sequential access is faster because of greater locality of references [Reason: Elements in contiguous memory locations]	Sequential access is slow because of low locality of references [Reason: Elements not in contiguous memory locations]

Time Complexity: Singly-linked lists vs. 1D-arrays

Operation	ID-Array Complexity	Singly-linked list Complexity
Insert at beginning		
Insert at end		
Insert at middle*		
Delete at beginning		
Delete at end		
Delete at middle*		
Search		
Indexing: What is the element at a given position k?		

Time Complexity: Singly-linked lists vs. 1D-arrays

Operation	ID-Array Complexity	Singly-linked list Complexity
Insert at beginning	Θ(n)	Θ(1)
Insert at end	Θ(1)	Θ(1) if the list has **tail** reference Θ(n) if the list has no **tail** reference
Insert at middle*	O(n)	O(n)
Delete at beginning	Θ(n)	Θ(1)
Delete at end	Θ(1)	Θ(n)
Delete at middle*	O(n): Θ(1) access followed by O(n) shifts	O(n): O(n) search, followed by Θ(1) delete
Search	O(n) linear search O(log n) Binary search	O(n)
Indexing: What is the element at a given position k?	Θ(1)	O(n)

*middle: neither at the beginning nor at the end

Doubly Linked Lists

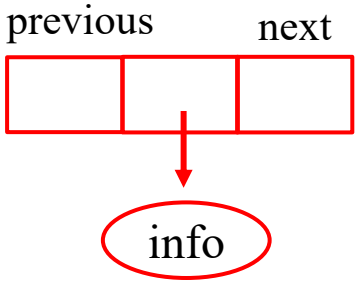
Doubly Linked Lists

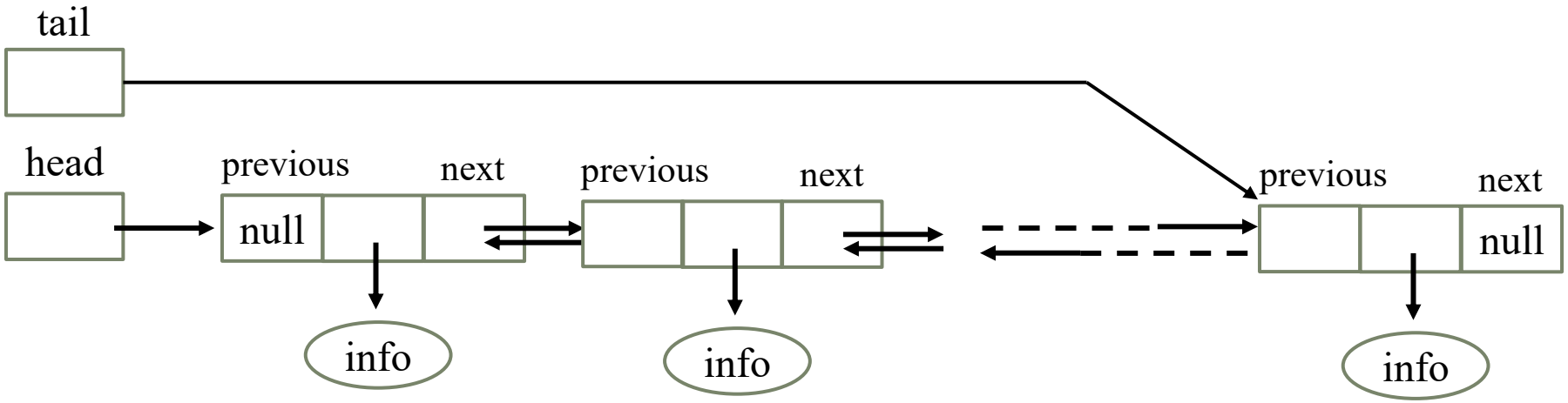
- What is a doubly-linked list?
- Doubly Linked Lists
  - Representation
  - Creation
  - Append and Prepend
  - Deletion
  - Traversal
  - Space Analysis
- Doubly-linked lists vs. Singly-linked lists
- Uses of doubly-linked lists

What is a Doubly-linked list?

- A doubly linked list is a dynamic data structure consisting of a sequence of nodes, forming a linear ordering.
- Each node stores:
  - info (data object)
  - next (reference i.e., address) to the next node
  - previous (reference i.e., address) to the previous node

Node:





Representation

```
public class DLL<T> {

    private static class DLLNode<T> {
        public T info;
        public DLLNode<T> next, prev;

        public DLLNode() {
            this(null,null,null);
        }

        public DLLNode(T e) {
            this(e,null,null);
        }

        public DLLNode(T el, DLLNode<T> n, DLLNode<T> p) {
            info = el; next = n; prev = p;
        }

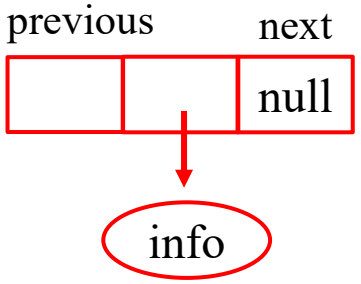
    }

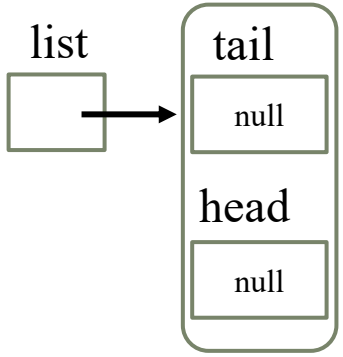
    private DLLNode<T> head, tail;

    public DLL() {
        head = tail = null;
    }

    public boolean isEmpty() {
        return head == null;
    }

    public void clear() {
        head = tail = null;
    }
```



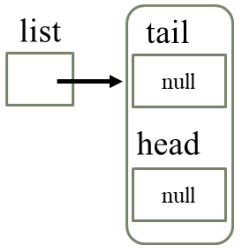


Empty list

List Creation and Insertion

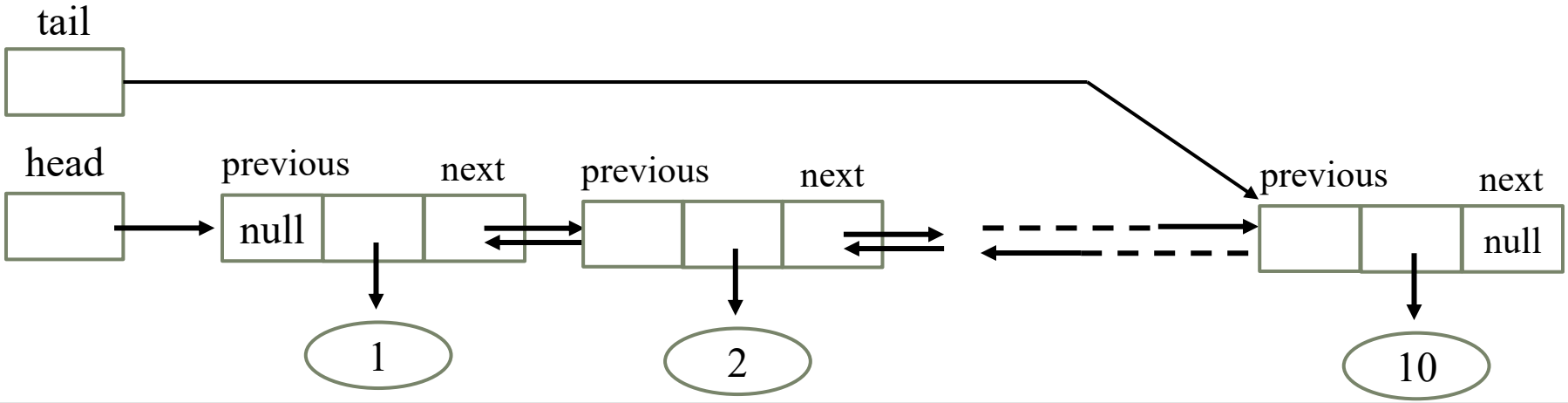
- An example of creating a doubly-linked list:

```
DLL<Integer> list = new DLL<>();
```



- Once created, elements can be inserted into the list using either the addToTail or addToHead methods:

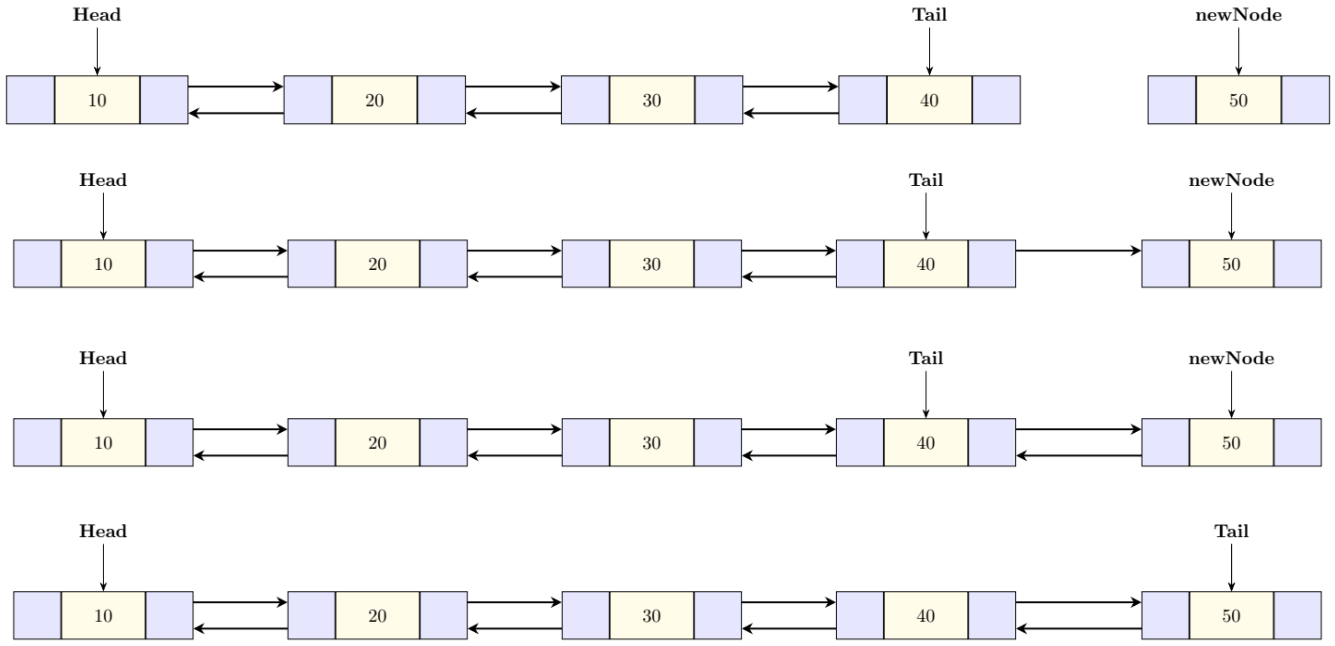
```
for(int k = 1; k <= 10; k++)
    list.addToTail(k);
```



Insertion at the end (append)

```java
public void addToTail(T e) {
    DLLNode<T> newNode = new DLLNode<T>(e);
    if (isEmpty()) // or if tail == null
        head = tail = newNode;
    else{
        tail.next = newNode;
        newNode.prev = tail;
        tail = newNode;
    }
}
```

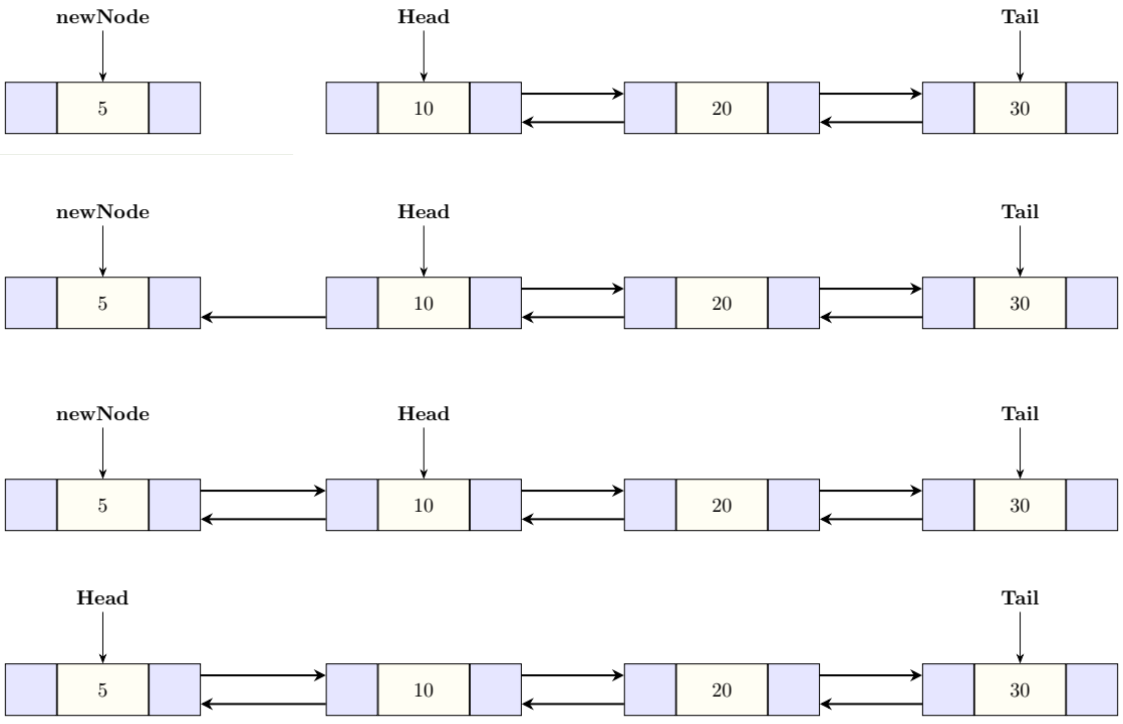
Complexity is



Insertion at the beginning (prepend)

```java
public void addToHead(T e){
    DLLNode<T> newNode = new DLLNode<T>(e);
    if(isEmpty())
        head = tail = newNode;
    else{
        newNode.next = head;
        head.prev = newNode;
        head = newNode;
    }
}
```

Complexity is



Insertion after node that is not last node

```
public void insertAfter(T e,T e1){
    DLLNode<T> newNode = new DLLNode<T>(e);
    DLLNode<T> tmp = head;

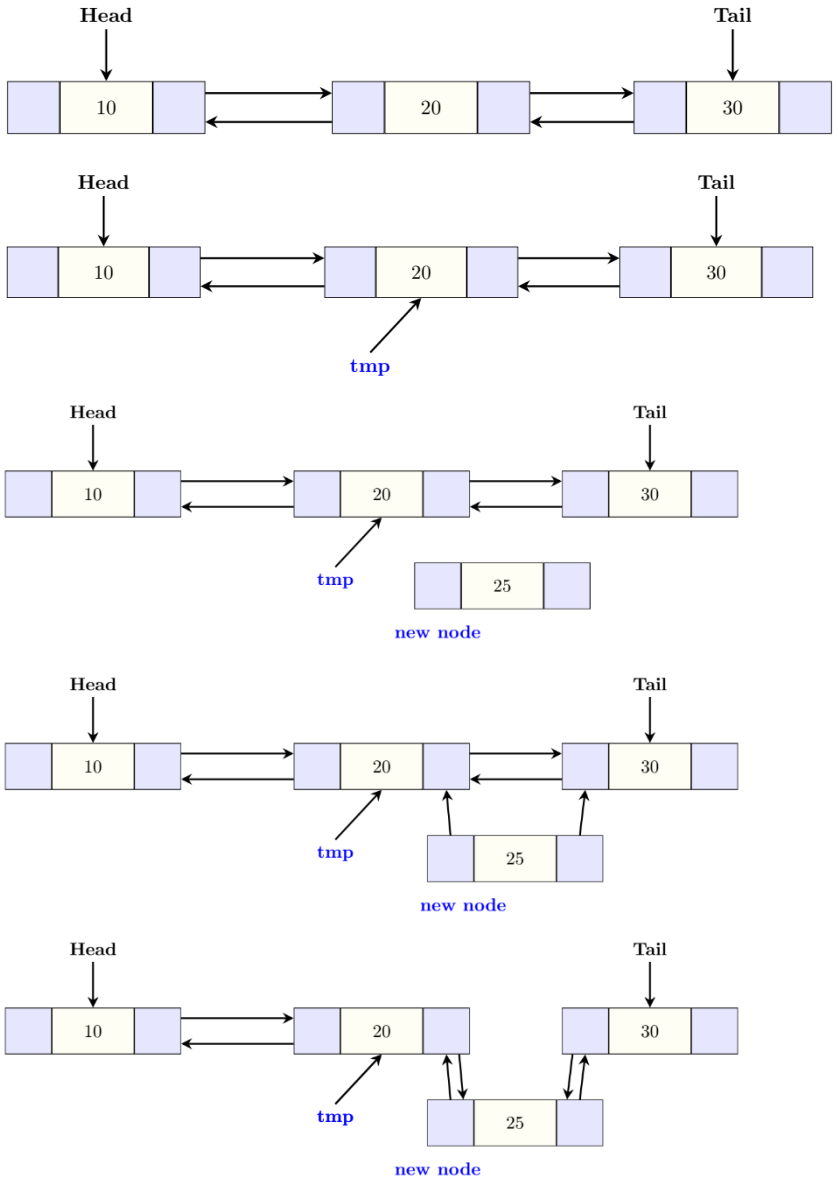
    while(!e1.equals(tmp.info))
        tmp = tmp.next;

    newNode.next = tmp.next;
    newNode.prev = tmp;

    tmp.next = newNode;
    newNode.next.prev = newNode;
}
```

Complexity is

Assume the node with info e1 exists and is not the last node.



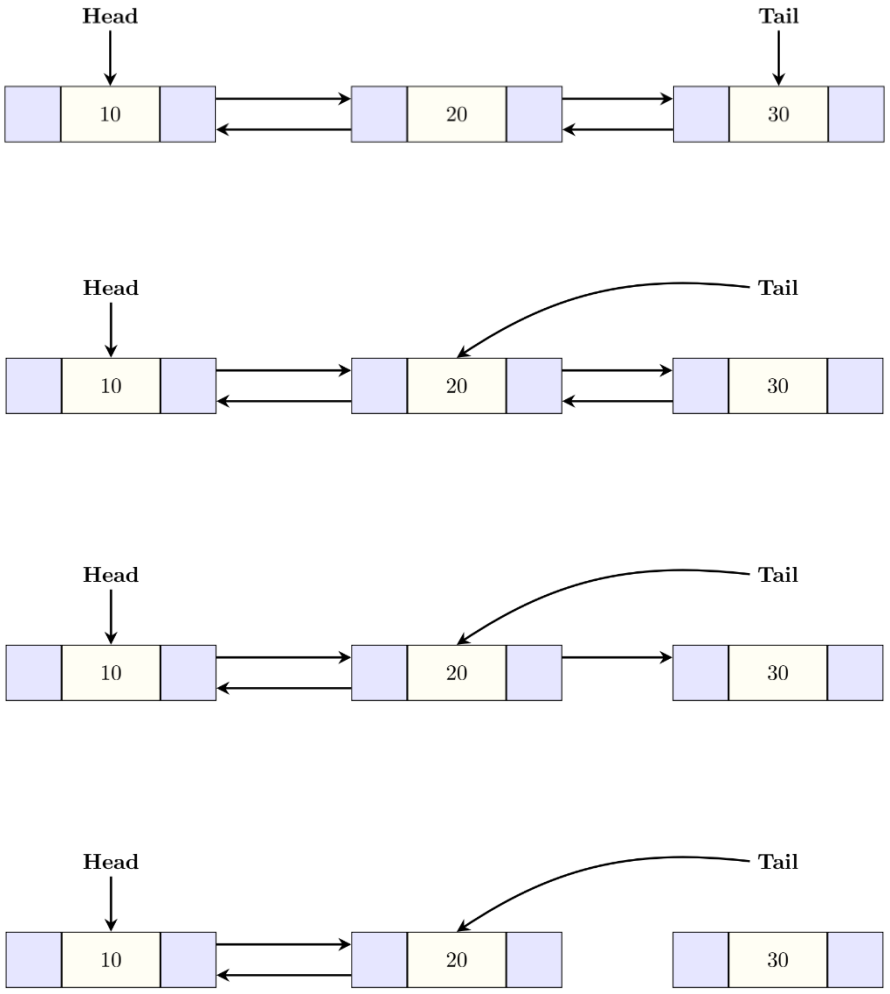
Deletion at the end

```
public T deleteFromTail() {
    // if list is empty;
    if (isEmpty())
        return null;

    T e = tail.info;
    // if only one node on the list;
    if (head == tail)
        head = tail = null;

    // if more than one node in the list;
    else {
        tail = tail.prev;
        tail.next.prev = null; //optional
        tail.next = null;
    }

    return e;
}
```



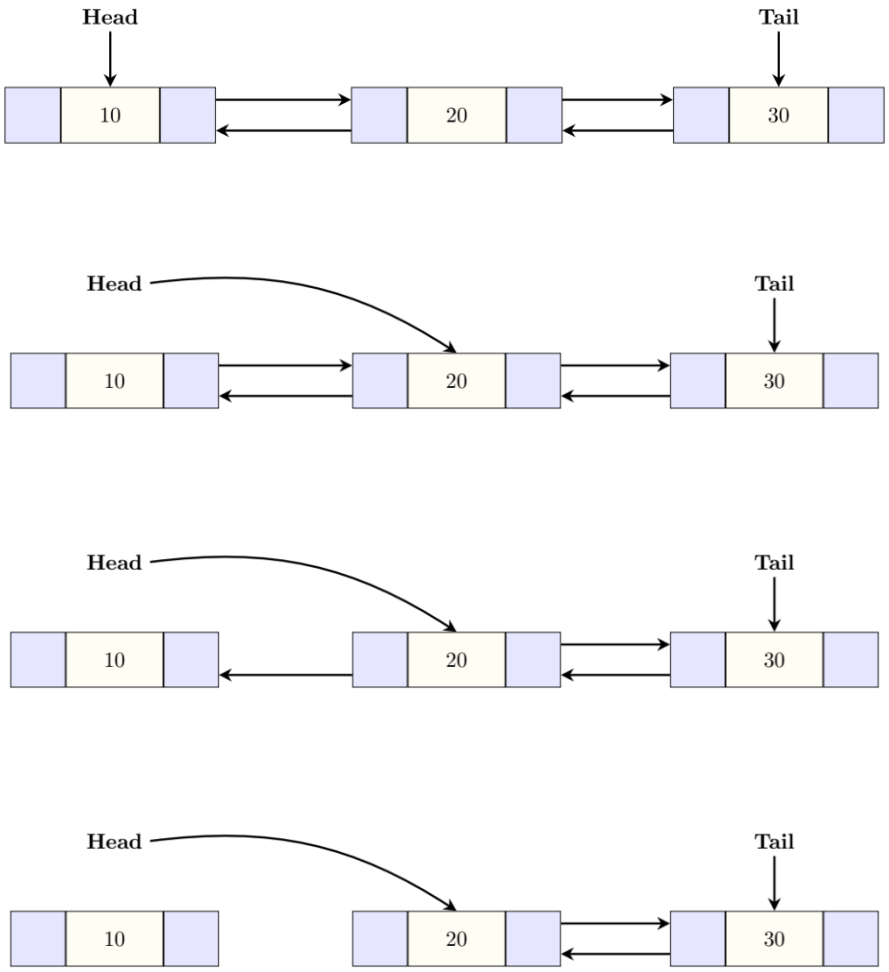
Complexity is

Deletion at the beginning

```
public T deleteFromHead() {
    // if list is empty;
    if (isEmpty())
        return null;

    T e = head.info;

    // if only one node on the list;
    if (head == tail)
        head = tail = null;
    // if more than one node in the list;
    else {
        head = head.next;
        head.prev.next=null;// optional
        head.prev = null;
    }
    return e;
}
```



Complexity is

Traversal

- For the DoublyLinked list, traversal can be done in either direction.
- Forward, starting from the head, or backward starting from the tail.

```
DLLNode<T> tmp = head;
while(tmp != null) {
  //do something
   tmp = tmp.next;
}
```

```
DLLNode<T> tmp = tail;
while(tmp != null){
  //do something
    tmp = tmp.prev;
}
```

- Example: Print a doubly linked list.

```
public void printList(){
 DLLNode<T> tmp = head;
 while(tmp != null){
    System.out.print(tmp.info + " ");
    tmp = tmp.next;
 }
}
```

Deleting a node that is neither the first nor the last

```
public void deleteNode(T e1){

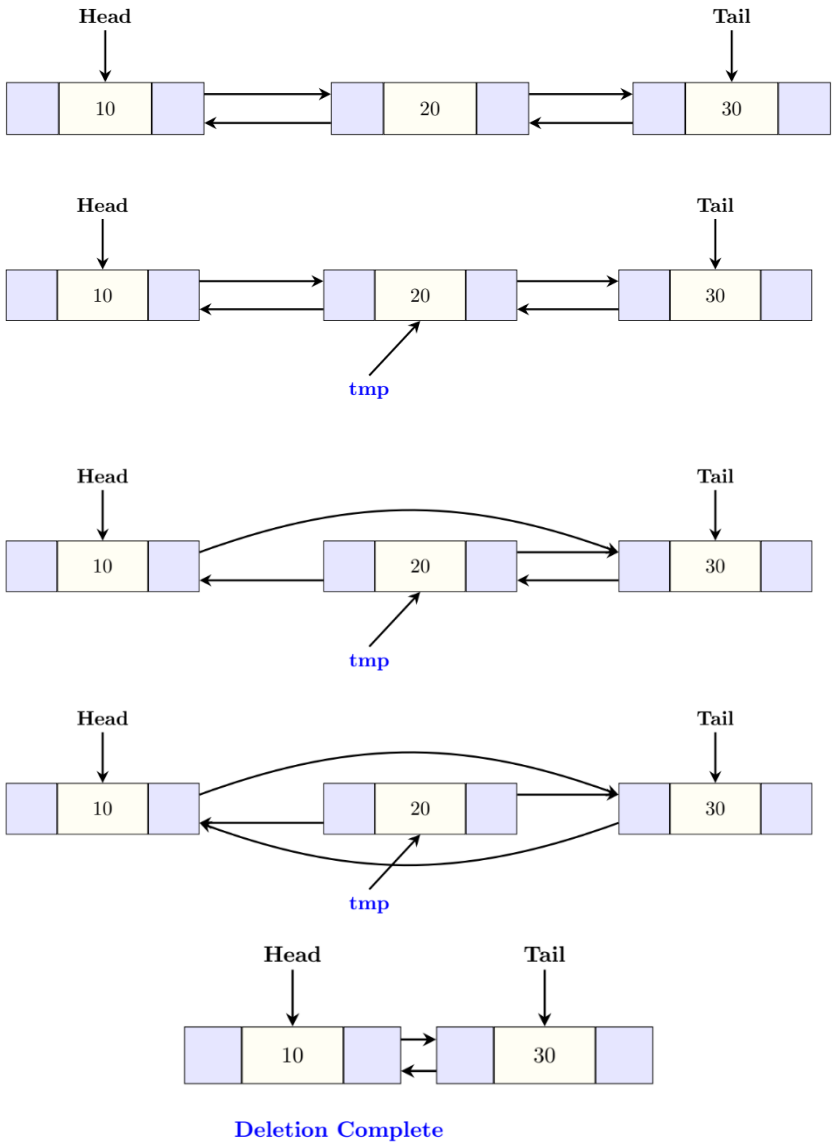
    DLLNode<T> tmp = head;

    while(! e1.equals(tmp.info))
        tmp = tmp.next;

    tmp.prev.next = tmp.next;
    tmp.next.prev = tmp.prev;
    tmp.next = null; // optional
    tmp.prev = null; // optional
}
```

Complexity is

Assume the node with info e1 exists and is neither the first node nor the last.



Doubly-Linked Lists: Space Requirements

- Each node in a doubly linked list stores:
  - **Data** for each node
  - A **pointer/reference** to the next node
  - A **pointer/ reference** to the previous node
- Number of references in the list and space required

Reference	Total
head	1
tail	1
next	n
Previous	n
info	n

So, each node uses slightly more memory than a singly linked list, but the total memory still grows **linearly** with the number of nodes:

$$\text{Total } Space = n \times (info + 2\ reference) \Rightarrow O(n)$$

The space complexity of a singly linked list with is O(n)

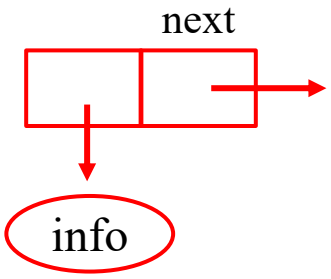
Doubly-Linked Lists vs. Singly-linked lists

- A doubly-linked list allows traversing the list in either direction.
- Modifying a doubly-linked list usually requires changing more references but is sometimes simpler because there is no need to keep track of the address of the previous node.
  - In a singly-linked list, this is required in delete and insert before operations.
- The deleteLast operation is
  - $\Theta(1)$ in doubly-linked list whereas
  - $\Theta(n)$ is singly-linked list
- A singly-linked list uses less memory than an equivalent doubly-linked list.

What is a Circular-linked list?

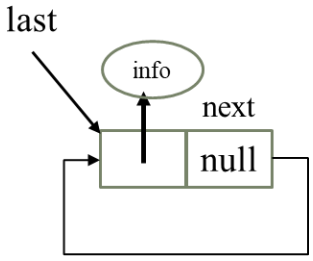
- A circular linked list is a sequence of nodes in which every node has a link to its next node in the sequence and the last node has a link to the first node.
  - Each node stores:
  - info (data object)
  - next (reference i.e., address) to the next node

Node:

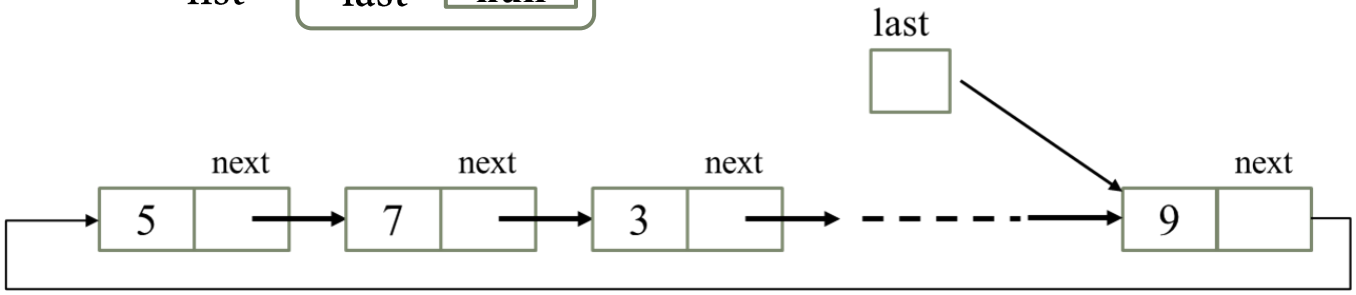


- In a circular singly linked list, a reference last always refers to the last node in the list; last is null if the list is empty:
- Empty circular linked list





A circular list with one node



Representation

```
public class CSLL<T> {

    private class SLLNode<T> {

        private T info;
        private SLLNode<T> next;

        public SLLNode() {
            this(null, null);
        }

        public SLLNode(T e) {
            this(e, null);
        }

        public SLLNode(T e, SLLNode<T> ptr) {
            info = e;
            next = ptr;
        }
    }

    protected SLLNode<T> last;

    public CSLL(){
        last = null;
    }

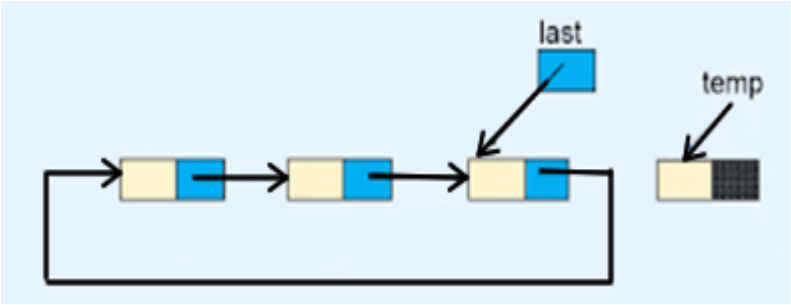
    public boolean isEmpty(){
        return last == null;
    }
```

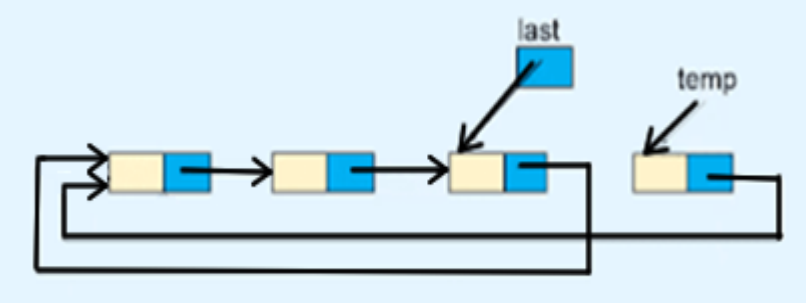
- An example of creating a circular singly-linked list:

```
CSLL<Integer> cll = new CSLL<Integer>();
cll.insertAtBeginning(13);
cll.insertAtBeginning(7);
```

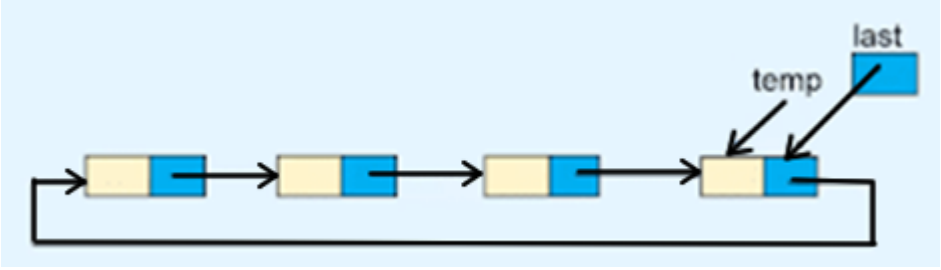
Insertion at the end

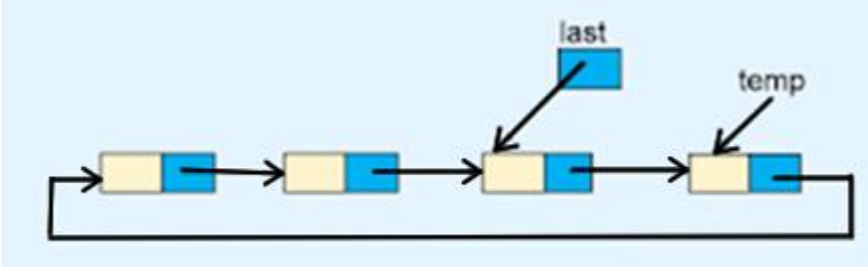
```
public void insertAtEnd(T e) {
    SLLNode<T> temp = new SLLNode<T>(e);
    if (isEmpty()){
        last = temp;
        temp.next = last;
    }
    else{
        temp.next = last.next;
        last.next = temp;
        last = temp; //
    }
}
```





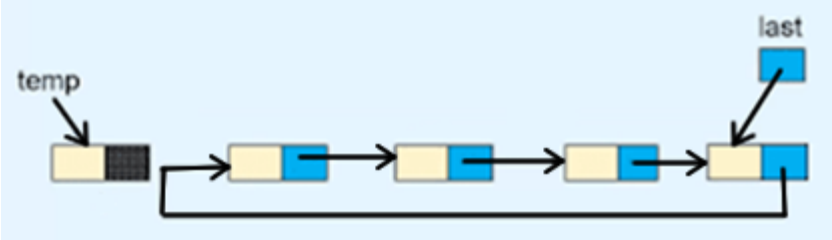
Complexity is

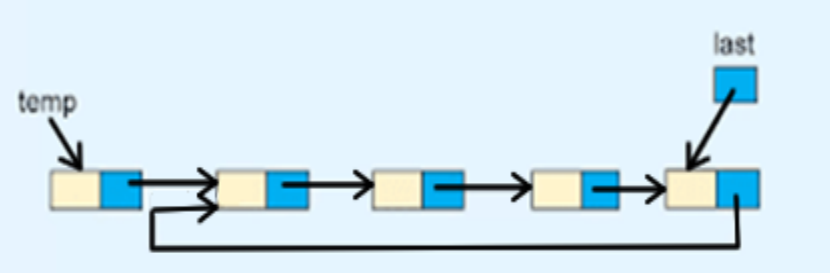




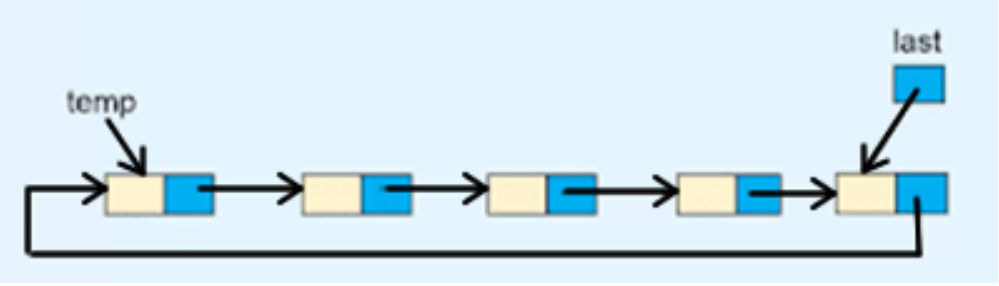
Insertion at the beginning

```
public void insertAtBeginning(T e){
    SLLNode<T> temp = new SLLNode<T>(e);
    if (isEmpty()){
        last = temp;
        temp.next = last;  }
    else{
        temp.next = last.next;
        last.next = temp;
    }
}
```





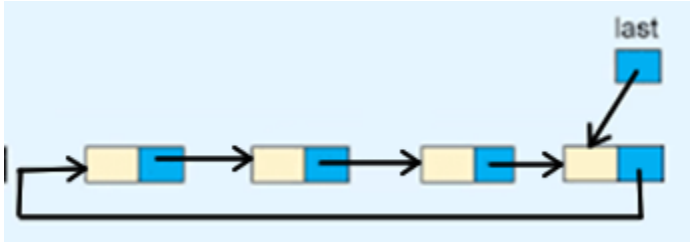
Complexity is

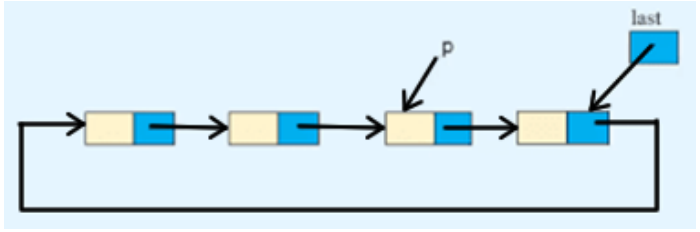


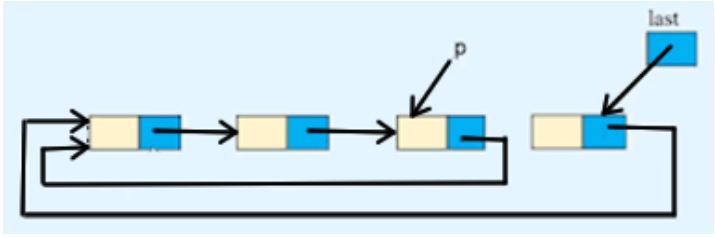
Deletion at the end

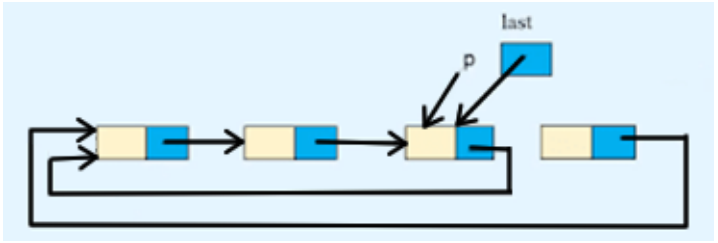
```java
public T deleteLastNode(){
    // if list is empty
    if(isEmpty())
        return null;
    T e = last.info;
    // if there is only one node in the list
    if(last == last.next)
        last = null;
    else{
        // More than one node in the list
        SLLNode<T> p = last.next;
        while(p.next != last){
            p = p.next;
        }
        p.next = last.next;
        last = p;
    }

    return e;
}
```









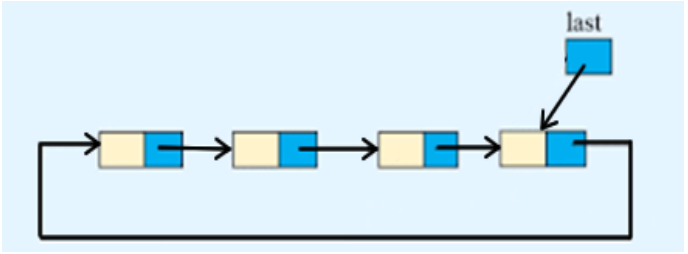
Complexity is

Deletion at the beginning

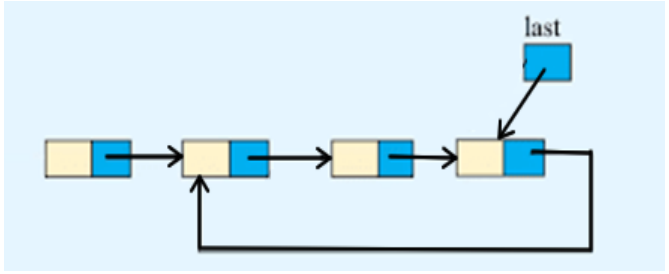
```
public T deleteFirstNode(){
    // if list is empty
    if(isEmpty())
        return null;

    T e = last.next.info;
    // if there is only one node in the list
    if(last == last.next)
        last = null;
    else
        // More than one node in the list
        last.next = last.next.next;

    return e;
}
```







Complexity is

Traversal

- Example: print a circular list by using toString() method:

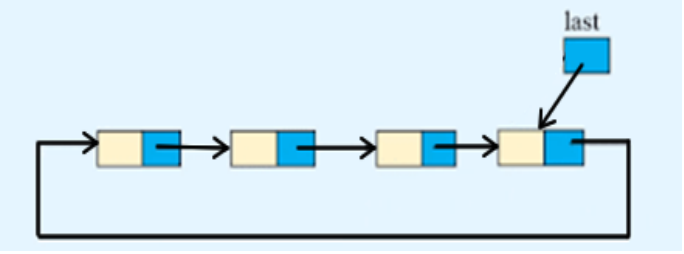
```
SLLNode<T> tmp =  last.next;


while(tmp != last){
    // process node

    tmp = tmp.next;
}
```

```
public String toString() {
    if(isEmpty())
        return "[ ]";
    String str = "[ ";
    SLLNode<T> tmp = last.next;
    do {
        str += tmp.info + "   ";
        tmp = tmp.next;
    } while (tmp != last.next);
    return str + "]";
}
```

Complexity is



Circular Linked List Variants

Operation	With Head Only	With Tail Only
Add at beginning	O(n)	O(1)
Add at end	O(n)	O(1)
Add/delete at middle	O(n)	O(n)
Delete head	O(n)	O(1)
Delete at tail	O(n)	O(n)

Comparison of singly-linked list and circular list

Operation	Singly linked list Complexity	Circular Singly-linked list Complexity
Insert at beginning		
Insert at end		
Insert at middle*	O(n)	O(n)
Delete at beginning		
Delete at end	O(n)	O(n)
Delete at middle*		
Search	O(n)	O(n)
Indexing: What is the element at a given position k?		

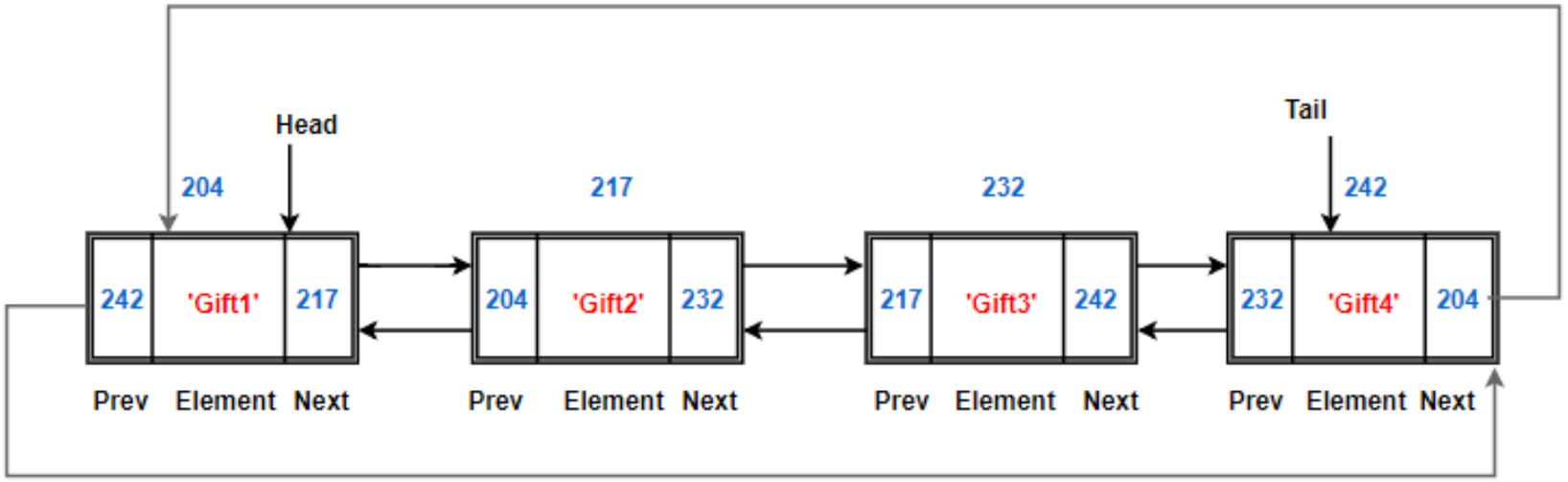
Comparison of singly-linked list and circular list

Operation	Singly linked list Complexity	Circular Singly-linked list Complexity
Insert at beginning	Θ(1)	Θ(1)
Insert at end	Θ(1) if the list has **tail** reference Θ(n) if the list has no **tail** reference	Θ(1)
Insert at middle*	O(n)	O(n)
Delete at beginning	Θ(1)	Θ(1)
Delete at end	Θ(n)	Θ(n)
Delete at middle*	O(n)	O(n)
Search	O(n)	O(n)
Indexing: What is the element at a given position k?	O(n)	O(n)

*middle: neither at the beginning nor at the end

Circular doubly-linked lists

- The implementation of a circular doubly-linked list is left as an exercise.



Applications of Linked Lists

- Implementation of other data structures.
  - e.g. stacks, queues, graphs, hash tables, etc.
- Operating systems:
  - memory allocation and CPU scheduling
- Polynomial Representation
- Representing sparse matrices
- Undo/Redo functionality
- Previous and next page in a web browser
- Music Player and Image viewer

Summary

- A linked list is a data structure composed of nodes, each node holding some information and a reference to another node in the list.
- A node in a singly linked list has a link only to its successor in this sequence.
- A node in a doubly linked list has links to its successor and predecessor.
- A circular list is when nodes form a ring: The list is finite and each node has a successor.
- The advantage of arrays over linked lists is that they allow random access.